

User Guide

The assembler in use: command line, source dialects, platforms, output formats, dialect conversion and the REST API — and the complete reference of the rat dialect, every CPU, platform and option

13 November 2026

Contents

Part I — Using rat 4

1 Introduction 5

1.1 What rat does . . . 5
1.2 Supported CPUs . . . 5
1.3 Three ways to use it . . . 6

2 Installation and quick start 7

2.1 Prebuilt programs . . . 7
2.2 Verifying downloads . . . 7
2.3 Building from source . . . 8
2.4 A first program . . . 8

3 The command line 9

3.1 Assembling: rat build . . . 9
3.2 Listing and label files . . . 9
3.3 Disassembling: rat disasm . . . 10
3.4 Converting: rat convert . . . 10
3.5 Serving: rat serve . . . 10

4 Source dialects 11

4.1 Which dialect a file uses . . . 11
4.2 The same program in three dialects . . . 11

5 Platforms 13

5.1 Overview . . . 13
5.2 Platform directives . . . 14

6 Machine by machine 15

6.1 Commodore: C64, C128, VIC-20, PET, C16/Plus/4, MEGA65 . . . 15
6.2 Atari 8-bit and Atari 2600 . . . 15
6.3 NES and SNES . . . 15
6.4 Z80 machines: CPC, ZX81, Spectrum, ColecoVision, MSX, MSX2, TRS-80 . . . 16
6.5 Acorn: BBC Micro, BBC Master, Archimedes . . . 17
6.6 Apple II and Macintosh . . . 18
6.7 Amiga . . . 18
6.8 Vectrex, TI-99/4A, IMSAI 8080 . . . 18

7 Working with mimic 20

8 Converting between dialects 21

9 REST API 22

Part II — Reference 23

10 The rat dialect 24
10.1 Labels and symbols . . . 24
10.2 Numbers, strings, operators . . . 24
10.3 Data . . . 25
10.4 Program counter and memory . . . 25
10.5 Control flow and reuse . . . 25
10.6 CPU, platform and target directives . . . 26
11 Operand syntax per CPU 27
12 CPUs 28
13 Platforms 29
13.1 Platform directives and functions . . . 30
14 Output formats 32
15 Command line reference 33
15.1 rat build SOURCE . . . 33
15.2 rat disasm FILE . . . 33
15.3 rat convert FILE . . . 33
15.4 rat serve . . . 33
16 REST API reference 34
16.1 POST /assemble . . . 34
16.2 POST /disassemble . . . 34
16.3 POST /convert . . . 34
16.4 GET /cpus, /platforms, /formats, /health . . . 35
17 Known gaps 36
18 How it works 37
18.1 Passes . . . 37
18.2 Amiga relocation . . . 37
18.3 Testing . . . 37
18.4 Building this guide . . . 37

Part I — Using rat

Installation, the command line, source dialects, platforms, examples for the machines, conversion and the REST API

1 Introduction

rat assembles text. It is a multi-CPU, multi-dialect assembler written in Rust, built to produce binaries for every CPU that the mimic emulator runs. One program covers the 6502 family, the Z80, the 68000 and the other processors of the classic home computers and consoles, and writes the file formats these machines load: PRG, D64, cartridge ROMs, tape and disk images.

This guide describes how to use the assembler: installing it, building programs on the command line, choosing a source dialect and a platform, writing the output formats, converting sources between dialects and using the REST API. Part II is the reference of the rat dialect, the CPUs, the platforms, the formats and every option.

1.1 What rat does

- **Assembles** ACME, KickAssembler and rat sources, also mixed in one project.
- **Targets 16 CPUs** with their native operand syntax (MOS, Zilog, Intel, Motorola, Sony, TI, Acorn).
- **Knows 30 platforms**: each one sets the CPU, the origin, the text encoding, the output formats and its own directives.
- **Writes 19 output formats**, from a raw binary to a D64 disk image or an Amiga hunk executable.
- **Disassembles** binaries of every CPU into rat syntax.
- **Converts** sources between the three dialects.
- **Serves a REST API** with the same functions, for editors and web tools.

1.2 Supported CPUs

CPU	Ids (`--cpu`, `.cpu`)	Byte order	Used by
MOS 6502 family	`6502` (with undocumented opcodes), `6502ni`, `6510`, `2a03`, `6507`	little	C64, C128, VIC-20, PET, C16/Plus/4, Atari 8-bit, Atari 2600, NES, Apple II, BBC Micro
CMOS 65C02	`65c02` (no Rockwell bit instructions)	little	Apple IIe enhanced, BBC Master (65SC12)
MEGA65 45GS02	`45gs02` (4510/65CE02 with Q instructions, flat pointers, long branches)	little	MEGA65
WDC 65C816	`65816`	little	SNES
Sony SPC700	`spc700`	little	SNES audio
Zilog Z80 (with undocumented instructions)	`z80`	little	Amstrad CPC, ZX81, ZX Spectrum, ColecoVision, MSX, MSX2, TRS-80, C128
Intel 8080	`8080` (Intel mnemonics)	little	IMSAI 8080
TI TMS9900	`tms9900`	big	TI-99/4A
ARM2	`arm2` (26-bit, Acorn mnemonics)	little	Acorn Archimedes
Motorola 6809	`6809`	big	Vectrex
Motorola 68000 / 68020	`68000`, `68020`	big	Amiga 500 to 1200, Macintosh

The encoding tables are ported from mimic's CPU cores. The tests check every opcode against mimic's disassemblers (about 180,000 instructions), so what rat assembles is what mimic runs.

1.3 Three ways to use it

Use	How
Command line	`rat build`, `rat disasm`, `rat convert` and the list commands `rat cpus`, `rat platforms`, `rat formats`.
REST API	`rat serve` starts an HTTP server with `/assemble`, `/disassemble` and `/convert`; OpenAPI description and Swagger UI included.
Library	The Rust crates `rat-asm`, `rat-cpu`, `rat-platform` and `rat-output` can be used from other Rust programs.

2 Installation and quick start

2.1 Prebuilt programs

Prebuilt programs for Linux, macOS and Windows are attached to each GitHub release (github.com/guidobonerz/rat/releases). Download the archive for your system and unpack it; `rat` is a single file and needs no installation.

Linux

```
tar -xzf rat-v0.1.0-x86_64-unknown-linux-gnu.tar.gz
cd rat-v0.1.0-x86_64-unknown-linux-gnu
chmod +x rat                      # only needed if the executable bit got lost
sudo mv rat /usr/local/bin/       # or ~/.local/bin, if that is on your PATH
rat --help
```

Use the `aarch64-unknown-linux-gnu` archive on ARM64 machines such as a Raspberry Pi 4/5 with a 64-bit system.

macOS

Pick `aarch64-apple-darwin` for Apple Silicon (M1 and later) and `x86_64-apple-darwin` for Intel Macs.

```
tar -xzf rat-v0.1.0-aarch64-apple-darwin.tar.gz
cd rat-v0.1.0-aarch64-apple-darwin
chmod +x rat
xattr -d com.apple.quarantine rat # the binary is not signed or notarized
sudo mv rat /usr/local/bin/
rat --help
```

Without the `xattr` step, macOS blocks the program with a message that the developer cannot be verified. As an alternative, try to run it once, then allow it under *System Settings → Privacy & Security → Open Anyway*.

Windows

Unpack the zip, for example to `C:\Tools\rat`, and add that folder to your `PATH` so that `rat` works in every terminal:

```
setx PATH "%PATH%;C:\Tools\rat"
```

Open a new terminal and run `rat --help`. If SmartScreen warns about an unrecognised app, click *More info → Run anyway*. If Windows marks the downloaded file as blocked, run this in PowerShell:

```
Unblock-File C:\Tools\rat\rat.exe
```

> **Note.** The Linux and Windows programs are compressed with UPX. Some antivirus tools flag UPX-packed files as suspicious. If yours quarantines `rat.exe`, check it against `SHA256SUMS.txt` and add an exclusion, or build from source.

2.2 Verifying downloads

Each release contains `SHA256SUMS.txt`:

```
sha256sum -c SHA256SUMS.txt --ignore-missing          # Linux
shasum -a 256 -c SHA256SUMS.txt --ignore-missing      # macOS
Get-FileHash rat-v0.1.0-x86_64-pc-windows-msvc.zip    # Windows, compare by hand
```

2.3 Building from source

- **Rust** (stable toolchain) with `cargo` — install it from rustup.rs.
- The **mimic** repository checked out next to rat, in a folder named `mimic`: the tests use its CPU cores and media readers.

```
cargo install --path crates/rat-cli       # puts rat in ~/.cargo/bin
cargo build --release -p rat-cli          # or: target/release/rat
```

2.4 A first program

Save this as `hello.asm` (KickAssembler dialect, the platform defaults to the C64):

```
BasicUpstart2(start)

        *= $0810 "Main"
start:  ldx #0
!loop:  lda text,x
        beq !done+
        jsr $ffd2
        inx
        bne !loop-
!done:  rts

text:   .encoding "petscii_mixed"
        .text "hello world"
        .byte 13, 0
```

Assemble it and start the result in mimic:

```
rat build hello.asm                       # -> hello.prg
mimic-desktop --machine c64 --prg hello.prg
```

The same idea in the rat dialect for another machine, here a ZX Spectrum tape:

```
; hello.rat
.platform spectrum
.tap name="HELLO", exec=start
.org $8000
start:  ld a, 2
        call $1601          ; CHAN-OPEN (screen)
        ld hl, msg
loop:   ld a, (hl)
        or a
        ret z
        rst $10             ; print
        inc hl
        jr loop
msg:    .text "Hello Spectrum!", 13
        .byte 0
```

```
rat build hello.rat                       # -> hello.tap, with a BASIC loader
```

3 The command line

`rat` has one command per task. `rat --help` and `rat <command> --help` print the options.

Command	Purpose
`rat build SOURCE`	Assemble a source file.
`rat disasm FILE --cpu CPU`	Disassemble a binary into rat syntax.
`rat convert FILE --to DIALECT`	Convert a source file to another dialect.
`rat serve`	Run the REST API server.
`rat cpus`	List the CPUs with aliases, byte order and address width.
`rat platforms`	List the platforms with their CPU, formats, directives and functions.
`rat formats`	List the output formats with extension and platforms.

3.1 Assembling: rat build

```
rat build main.asm                      # dialect from the extension, c64 -> main.prg
rat build game.rat -p nes -o game.nes
rat build demo.a -D DEBUG=1 -I lib --listing demo.lst --vice-labels demo.vs
rat build intro.asm -f d64              # D64 with one PRG
rat build cart.rat -p msx2 -f bin       # MSX2 cartridge ROM
```

- The **dialect** follows the file extension (`.a` ACME, `.asm` KickAssembler, `.rat` rat) unless `--syntax` or a `.syntax` line says otherwise.
- The **platform** comes from `-p`, from a `.platform` line in the source, or from the dialect: KickAssembler and ACME sources start on `c64`, rat sources on `generic`.
- The **format** is the platform's first format unless `-f` or `.output` names another.
- The **output file** is the source name with the format's extension unless `-o` is given.
- `-D NAME` or `-D NAME=VALUE` defines a symbol (KickAssembler sources also see it in `cmdLineVars`); `-I DIR` adds an include directory. Both can be repeated.

Errors are printed in the style of the Rust compiler, with the file, the line and the source line marked. The exit code is not zero when the build failed.

3.2 Listing and label files

Option	Writes
`--listing FILE`	A listing: address, bytes, line number and source line.
`--vice-labels FILE`	VICE labels (`al C:xxxx .name`), which mimic's debugger and VICE load directly; `.break` and `.watch` in the source become breakpoints.
`--sym FILE`	A KickAssembler `.sym` file.
`--labels FILE`	An ACME label list.
`--json`	A summary of outputs, symbols, diagnostics and source lines as JSON on standard output, for build scripts and editors.

A listing looks like this:

```
004000  c9                          1  start: ret
004001  00 40                       2         .word start
```

3.3 Disassembling: rat disasm

```
rat disasm game.prg --cpu 6510          # the PRG load address is honoured
rat disasm rom.bin --cpu 6809 --org $e000
rat disasm code.bin --cpu 65816 --a16 --i16
```

The output is rat-dialect source that assembles to the same bytes. `--org` sets the load address, `--skip N` skips a header, `--prg` treats the file as a PRG whatever its name. For the 65816, `--a16` and `--i16` set the width of the accumulator and the index registers.

3.4 Converting: rat convert

```
rat convert old.a --to kick -o new.asm   # ACME -> KickAssembler
rat convert demo.asm --to rat            # to standard output
```

See the chapter *Converting between dialects* for what is carried over.

3.5 Serving: rat serve

```
rat serve                                # 127.0.0.1:8080
rat serve --bind 0.0.0.0:9000
```

See the chapter *REST API*.

4 Source dialects

rat reads three dialects and can convert between them.

Dialect	Extension	Character
ACME	`.a`	`!` pseudo-ops, labels without colons, `.local` and `@cheap` labels, anonymous `-`/`+` labels, `+macro` calls, `!zone`, `!for`, `!if`, `!do`, `!pseudopc`, `!convtab`, `!binary`, `!source`, `!to`. For 68000 sources also the Motorola data directives without `!`: `dc.b`/`dc.w`/`dc.l`, `ds.b`/`ds.w`/`ds.l count[, fill]` and `even`.
KickAssembler	`.asm`	`.` directives, `label:`, `!multi:` labels, `{ }` scopes and the whole script language: `.var`/`.const`/`.eval`, functions, macros, pseudocommands, lists, hashtables, structs, `.enum`, the math library, vectors and matrices, the `#import`/`#if` preprocessor, `.namespace`, segments, `.file`, `.disk`, `LoadBinary`/`LoadSid`/`LoadPicture`, `.assert`.
rat	`.rat`	rat's own dialect: CPU-neutral, Kick-style directives, `;` and `//` comments, plus `.org`, `.include`, `.incbin`, `.fn`, `.for i in a..b`, `.elif`, `dc.b`/`ds.w`, `@local` and `.local` labels. Part II documents it in full.

4.1 Which dialect a file uses

The first match wins:

- A `.syntax acme|kick|rat` (or `!syntax`) line at the top of the file.
- `--syntax` on the command line (main file only).
- The extension: `.a` is ACME, `.asm` is KickAssembler, `.rat` is rat.
- The dialect of the file that includes it.

Every included file is detected on its own, so one project can mix dialects: a KickAssembler main file can include an ACME library and a rat file with tables.

> **Note.** Instruction operands always use the CPU's native syntax (Zilog, Intel, Motorola, Sony, MOS, TI, Acorn), whatever the dialect. A Z80 instruction reads the same in an ACME, a KickAssembler and a rat source.

4.2 The same program in three dialects

ACME

```
!to "hello.prg", cbm
!cpu 6510
* = $c000
CHROUT = $ffd2
!zone main
start   ldx #0
.loop   lda .msg,x
        beq +
        jsr CHROUT
        inx
        bne .loop
+       rts
.msg    !pet "hello", 13, 0
```

KickAssembler

```
        *= $c000
.const CHROUT = $ffd2
start:  ldx #0
!loop:  lda msg,x
        beq !done+
        jsr CHROUT
        inx
        bne !loop-
!done:  rts
msg:    .encoding "petscii_mixed"
        .text "hello"
        .byte 13, 0
```

rat

```
.platform c64
.org $c000
CHROUT = $ffd2
start:  ldx #0
@loop:  lda msg,x
        beq @done
        jsr CHROUT
        inx
        bne @loop
@done:  rts
msg:    .petscii "hello"
        .byte 13, 0
```

5 Platforms

A **platform** chooses the default CPU, the default origin, the allowed output formats, the text encoding and the platform directives. Select it on the command line with `-p`, or in the source:

```
.platform c64        ; KickAssembler and rat
!platform nes        ; ACME
```

Features that belong to one platform only exist on that platform. `BasicUpstart`, `LoadSid`, `LoadPicture`, `.disk` and the formats `prg`, `d64` and `crt` are Commodore features. Using them on `nes`, `cpc` or `amiga` is an error that names the platform you need. KickAssembler and ACME sources default to `c64`, so existing sources build unchanged; rat sources start on `generic`.

> **Note.** The default origin applies to the platform a build starts with (from `-p` or the dialect). After a `.platform` line in the source, set the origin yourself with `.org`, as every example in this guide does.

5.1 Overview

Platform	CPU	Formats (first = default)	Platform directives
`c64`	6510	prg, d64, crt	`basicupstart`, `crt`, `disk`; `LoadSid()`, `LoadPicture()`
`c128`, `vic20`, `pet`, `plus4`	6502	prg, d64	`basicupstart`, `disk`; `LoadSid()`, `LoadPicture()`
`mega65`	45gs02	prg, d64	`basicupstart` (BASIC 65 `BANK 0:SYS`), `disk`; `LoadSid()`, `LoadPicture()`
`atari8`	6502	xex	`xex.run`, `xex.init`
`atari2600`	6507	a26	`a26.bank n`
`nes`	2a03	nes	`ines mapper= prg= chr= mirroring=`, `nes.chr { }`, `nes.bank n`
`snes`	65816	sfc	`snes.header title= map=lorom|hirom|exhirom`, `snes.spc [addr] { }`
`cpc`	z80	amsdos	`amsdos name= exec=`
`zx81`	z80	p	–
`spectrum`	z80	tap, sna	`tap name= exec= loader=`, `sna exec= sp= border= im= ei=`
`msx`, `msx2`	z80	bin (cartridge ROM), cas	`msx.header init`, `cas name= exec=`
`trs80`	z80	cas, cmd	`cas name= exec=`, `cmd exec`
`coleco`	z80	bin	`coleco.header start`
`imsai`	8080	bin	–
`bbc`, `bbcmaster`	6502, 65c02	ssd	`ssd name= exec= title= boot=`
`apple2`, `apple2e`	6502, 65c02	dsk	`dsk name= volume= type= locked=`
`mac`	68000	bin	–
`amiga`, `amiga1200`	68000, 68020	hunk (with RELOC32)	`amiga.section name, code|data|bss`
`vectrex`	6809	bin	`vectrex.header "TITLE"`
`ti99`	tms9900	bin (cartridge ROM at $6000)	`ti99.header "TITLE", start`
`archimedes`	arm2	bin	–
`generic`, `sim6502`	6502	bin	–

`bin`, `hex` (Intel HEX) and `srec` (Motorola S-records) are available on every platform. `rat platforms` and `rat formats` print the full lists.

5.2 Platform directives

A platform directive is written `.name …` while its platform is active, or `.<platform>.<name> …` anywhere. The second form also works when another platform is active:

```
.platform msx2
.header init            ; same as .msx.header init

.c64.basicupstart start
.xex.run start
```

Some directives have a short namespace of their own: `xex` (Atari 8-bit), `a26` (Atari 2600), `ines` (NES), `spc` (SNES), `amsdos` (CPC), `hunk` (Amiga), `tap` and `sna` (Spectrum), `dsk` (Apple II), `ssd` (BBC) and `cmd` (TRS-80).

6 Machine by machine

One short example for each group of machines. The sources are the ones of rat's test corpus (`tests/corpus`), so each of them builds as shown.

6.1 Commodore: C64, C128, VIC-20, PET, C16/Plus/4, MEGA65

`basicupstart` writes a BASIC line `10 SYS address` at the start of BASIC memory; the code continues after it. On the MEGA65 the line is `10 BANK 0:SYS address` at `$2001`. `LoadSid()` and `LoadPicture()` read SID tunes and pictures, `.disk` builds a D64 with several files, `.crt` sets the header of a C64 cartridge.

```
.platform mega65
.basicupstart start
.org $2100
start:  ldq $10             ; 45GS02: 32-bit Q register
        lda ($10),z
        sta [$10],z         ; flat 28-bit pointer
        rts
```

```
rat build intro.asm                     # PRG
rat build intro.asm -f d64              # D64 with the PRG as its only file
```

6.2 Atari 8-bit and Atari 2600

```
.platform atari8
.org $2000
start:  lda #$22
        sta $d01a
        jmp start
.xex.run start              ; RUNAD segment: start address after loading
```

```
.platform atari2600
.org $f000
reset:  sei
        jmp reset
.org $fffc
        .word reset, reset
```

`.a26.bank n` starts a 4 KB bank of an F8, F6 or F4 cartridge; the `a26` format writes 2 KB, 4 KB or the banked sizes.

6.3 NES and SNES

```
.platform nes
.ines mapper=0, prg=1, chr=1, mirroring=v
.org $c000
reset:  sei
        cld
        ldx #$ff
        txs
-       bit $2002
        bpl -
        jmp *
nmi:    rti
.org $fffa
        .word nmi, reset, nmi
.nes.chr {
        .fill 16, $ff
}
```

On the SNES, `.snes.header` writes the internal ROM header and the checksum is computed. `.snes.spc` assembles a block with the SPC700 for the sound processor and stores it at the current ROM position:

```
.platform snes
.snes.header title="RAT TEST", map=lorom
.org $008000
reset:  sei
        clc
        xce
        rep #$30
        .a16
        .i16
        lda #$1234
        jml reset
.snes.spc $0200 {
        mov a, #$10
        mov $f2, a
spin:   bra spin
}
```

6.4 Z80 machines: CPC, ZX81, Spectrum, ColecoVision, MSX, MSX2, TRS-80

```
; Amstrad CPC: binary with an AMSDOS header
.platform cpc
.amsdos name="HELLO.BIN", exec=start
.org $4000
start:  ld hl, msg
@loop:  ld a, (hl)
        or a
        ret z
        call $bb5a          ; TXT OUTPUT
        inc hl
        jr @loop
msg:    .text "Hello CPC!", 13, 10
        .byte 0
```

The ZX Spectrum writes a TAP file (with `exec=` a BASIC loader goes first) or a 48K snapshot (`sna`). The ZX81 writes a `.p` file.

MSX and MSX2

`msx` is the MSX1, `msx2` the MSX2 that mimic emulates as the model `msx2` (V9938 video chip, memory mapper, sub ROM, clock). Both have the same CPU, the same cartridge header and the same cassette format, so a source moves between them by changing the platform line. A cartridge that uses the V9938 or the sub ROM belongs on `msx2`.

```
; a 16 KB cartridge at $4000
.platform msx2
.org $4000
.header init                ; "AB", INIT address, 12 zero bytes

CHPUT = $00a2

init:   ld hl, text
@loop:  ld a, (hl)
        or a
        jr z, @halt
        call CHPUT
        inc hl
        jr @loop
@halt:  di
        halt
text:   .asciiz "Hello from rat"
        .fill $8000 - *, $ff
```

```
rat build cart.rat                      # -> cart.bin
mimic-desktop --machine msx --model msx2 --cart cart.bin
```

A cassette file for `BLOAD"CAS:",R` is written with the `cas` format:

```
.platform msx2
.cas name="HELLO", exec=main
.org $c000
main:   jr main
```

TRS-80

```
; Model I: a SYSTEM tape (SYSTEM, HELLO, /)
.platform trs80
.cas name="HELLO", exec=start
.org $5200
start:  ld hl, text
        ld de, $3c00        ; video memory
        ld bc, textend - text
        ldir
@wait:  jr @wait
text:   .text "HELLO FROM RAT"
textend:
```

The `cmd` format writes a TRSDOS load module; `.cmd exec` sets its entry point.

6.5 Acorn: BBC Micro, BBC Master, Archimedes

```
; an SSD image that starts with Shift+BREAK
.platform bbc
.ssd title="RAT", boot=true, exec=start
OSWRCH = $ffee
.org $1900
start:  ldx #0
@loop:  lda text,x
        beq @done
        jsr OSWRCH
        inx
        bne @loop
@done:  rts
text:   .asciiz "Hello from rat"
```

`bbcmaster` selects the 65C02. The Archimedes uses the ARM2 with Acorn mnemonics and writes raw code:

```
.platform archimedes
.org $8000
        mov r0, #1
        stmfd r13!, {r0-r3, r14}
        bl sub
        ldmfd r13!, {r0-r3, pc}^
sub:    movs pc, r14
```

6.6 Apple II and Macintosh

```
; Apple IIe enhanced (65C02): DOS 3.3 disk with the program HELLO
.platform apple2e
.dsk name="HELLO", volume=254
.org $2000
start:  ldx #0
loop:   lda msg,x
        beq done
        jsr $fded           ; COUT
        inx
        bra loop
done:   rts
.encoding "apple"
msg:    .text "HELLO APPLE II", 13
        .byte 0
```

The `dsk` format is a DOS 3.3 image with the program as a binary file: boot a DOS and `BRUN` it. The Macintosh platform writes raw 68000 code.

6.7 Amiga

The `hunk` format is an executable for AmigaDOS. `.amiga.section` starts a hunk; absolute longwords that point into a hunk are relocated (RELOC32).

```
.platform amiga
start:  move.l  #msg, a0
        lea     sub(pc), a1
        move.w  #$4000, $dff09a
        moveq   #0, d0
.loop:  addq.l  #1, d0
        cmp.l   #100, d0
        bne.s   .loop
        jsr     sub
        rts
sub:    rts
.amiga.section data, data
msg:    dc.b "hello", 0
ptr:    dc.l start, msg
```

`amiga1200` selects the 68020 with its extra addressing modes and bit field instructions.

6.8 Vectrex, TI-99/4A, IMSAI 8080

```
.platform vectrex
.org 0
.vectrex.header "RAT TEST"
main:   jsr $f192
        lda #$7f
        ldx #table
        ldd ,x++
        lbra main
table:  .word $1234, main
```

```
; TI-99/4A: a cartridge ROM at $6000
.platform ti99
.org $6000
.header "RAT DEMO", start
start:  limi 0
        lwpi $8300
        li r1, text
loop:   movb *r1+, @$8c00
        jmp loop
text:   .text "HELLO FROM RAT"
        .even
```

```
.platform imsai
.org $0100
        mvi a, $2a
        out $ff             ; front panel lamps
        hlt
```

7 Working with mimic

rat and mimic are built for each other: rat writes the files mimic loads, and the label file lets the debugger show your names.

rat output	Load in mimic with
`prg`, `xex`, Spectrum `sna`	`--prg FILE`
`d64`, Apple `dsk`, BBC `ssd`	`--disk FILE`
`crt`, `nes`, `sfc`, `a26`, cartridge `bin` (MSX, ColecoVision, Vectrex, TI-99/4A)	`--cart FILE`
`tap`, `cas`, `p`	`--tape FILE`

```
rat build game.asm --vice-labels game.vs
mimic-desktop --machine c64 --prg game.prg --debug
```

- `--vice-labels` writes `al C:xxxx .name` lines. mimic's debugger and VICE load them directly.
- `.break` and `.watch` in the source add breakpoints to that file.
- `--listing` shows which bytes each source line produced, with the addresses the debugger shows.

mimic's own user guide describes the emulator's options, the media each machine accepts and the debugger.

8 Converting between dialects

`rat convert` parses a file and writes the same program in another dialect.

- Comments and blank lines are kept.
- ACME zones, `.local`/`@cheap` labels and anonymous labels are mapped onto KickAssembler's `{ }` scopes and `!` multi-labels, and back.
- Operators are re-spelled for the target, for example ACME `=`, `XOR` and `^` become `==`, `^` and `pow()`.
- Data byte order follows the CPU.

Constructs the target cannot express are reported as warnings and left as `TODO(rat convert)` comments. KickAssembler lists, structs and functions, for example, cannot be converted to ACME. The test suite converts every corpus file to every dialect and checks that the result assembles to identical bytes.

```
rat convert old.a --to kick -o new.asm
rat convert new.asm --to rat -o new.rat
rat convert tables.rat --from rat --to acme
```

9 REST API

`rat serve` starts an HTTP server (default `127.0.0.1:8080`). The API is described by OpenAPI at `/openapi.json`, with Swagger UI at `/docs`.

Method	Path	Purpose
POST	`/assemble`	Assemble a project: JSON or multipart/form-data.
POST	`/disassemble`	Disassemble bytes into rat syntax.
POST	`/convert`	Convert a source between the dialects.
GET	`/cpus`, `/platforms`, `/formats`	The lists the command line prints, as JSON.
GET	`/health`	Liveness check.

Requests only see the files they upload, through an in-memory file system; the server never touches its own disk. Bodies are limited to 32 MB and each assembly to 30 seconds.

```
curl -s localhost:8080/assemble -H 'content-type: application/json' \
  -d '{"files":{"main.asm":"BasicUpstart2(start)\nstart: inc $d020\n jmp start\n"},
       "main":"main.asm"}'
```

The response holds `ok`, the outputs (base64), the diagnostics with file, line, column and the rendered text, the symbols, the `.print` output, the listing and the VICE labels. Part II lists every field.

Part II — Reference

The rat dialect, operand syntax, CPUs, platforms, output formats, command line options, the REST API and known gaps

10 The rat dialect

rat's own dialect is meant for every CPU, not just the 6502. Use it with the extension `.rat`, or put `.syntax rat` at the top of the file. It reads like KickAssembler without the C64 assumptions, plus a few conveniences from ca65, vasm and Devpac.

```
; comments: `;` or `//`, block comments /* ... */
.platform nes                 ; CPU, formats, directives
.ines mapper=0, prg=1, chr=1, mirroring=v

.org $c000
reset:  sei
        ldx #$ff
        txs
@wait:  bit $2002             ; @local / .local: local to the last global label
        bpl @wait
-       jmp -                 ; anonymous labels like ACME
nmi:    rti
.org $fffa
        .word nmi, reset, nmi
```

10.1 Labels and symbols

Syntax	Meaning
`name:`	label at the program counter
`@name:` / `.name:`	cheap local label, visible until the next global label
`-` `--` `+` `++`	anonymous labels, referenced as `-`, `+`, … (as in ACME)
`name = expr`	constant label
`.const x = e`, `.var x = e`, `.set x = e`, `.label x = e`	constant, variable (`.eval x = …` changes it), label
`name: { … }`, `.scope name { … }`	named scope; inner labels are reached as `name.inner`
`.namespace ns { … }`	namespace

10.2 Numbers, strings, operators

- **Numbers:** `$ff`, `0xff`, `%1010`, `0b1010`, `0o17`, `123`, `1.5`, `'a'`.
- **Strings:** `"..."` with the escapes `\n \t \" \\ \0 \$xx`.
- **Operators** (C-like, loosest binding first): `?:`, then `||`, `&&`, `|`, `^`, `&`, then `== !=`, then `< <= > >=`, then `<< >> >>>`, then `+ -`, then `* / %`, then `**`, then unary `- ! ~`.
- **`<` and `>` at the start of an expression** take the low and high byte of everything after them: `#<table+1` is `lo(table+1)`.
- **`/`** is integer division when both sides are integers.
- **Program counter:** `*` or `$`.
- **Lists and indexing:** `[1, 2, 3]` builds a list; `list[i]` reads an element.
- **Math functions:** `lo hi bank`, `abs min max floor ceil round sqrt sin cos tan pow` and more.
- **String functions:** `toIntString toHexString toBinaryString`, `len`, `defined(name)`.
- **Constructors:** `List() Hashtable() Vector() Matrix()`.
- **Methods**, as in KickAssembler: `list.add() .get() .size()`, `hash.put() .get() .keys()`, `string.substring() .toUpperCase()` and more.

10.3 Data

Directive	Meaning
`.byte/.db`, `.word/.dw`, `.long/.dword`, `.quad`	in the CPU's byte order (`.endian le|be` overrides)
`.le16 .be16 .le24 .be24 .le32 .be32`	explicit byte order
`dc.b dc.w dc.l`, `ds.b ds.w ds.l n[, fill]`	68k style
`.text "…"`, `.asciiz "…"`, `.petscii "…"`, `.screen "…"`	text in the current encoding, NUL-terminated, PETSCII, C64 screen codes
`.encoding "name"`	text encoding (the default comes from the platform): `ascii`, `petscii_mixed`, `petscii_upper`, `screencode_mixed`, `screencode_upper`, `atari_screen`, `zx81`, `apple` (ASCII with bit 7 set, for Apple II `COUT`)
`.hex 0123 abcd`	hex bytes
`.fill n, expr`	`expr` may use `i`, the index (as in KickAssembler)
`.lohifill n, expr`	low and high byte tables, reached as `label.lo` / `label.hi`
`.res n` / `.ds n`	n zero bytes
`.align n[, fill]`, `.even`	padding
`.incbin "file"[, offset[, length]]`	binary include

10.4 Program counter and memory

Directive	Meaning
`.org addr`, `*= addr`, `.pc = addr "block name"`	set the program counter (starts a new memory block)
`.pseudopc addr { … }` or `.pseudopc addr … .realpc`	assemble for another address
`.segmentdef`, `.segment`, `.segmentout`, `.file`, `.memblock`	KickAssembler segments
`.output "file"[, format]`	main output file and format

10.5 Control flow and reuse

```
.if x > 3 { … } .elif x { … } .else { … }
.ifdef NAME { … } .else { … }
.for i in 0..8 { .byte i * 2 }          ; exclusive; 0..=8 is inclusive; `step n`
.for v in [1, 5, 9] { .byte v }
.while n < 10 { .eval n = n + 1 }
.repeat 4 { nop }
.macro poke(addr, value = 0) { lda #value
                               sta addr }
poke($d020, 1)        ; or: poke $d020, 1
.fn double(x) = x * 2
.fn clamp(x) {
    .if x > 255 { .return 255 }
    .return x
}
.pseudocommand mov src : dst { lda src
                               sta dst }
mov #1 : $d020
.struct Point {x, y}
.enum {RED, GREEN = 5, BLUE}
#define DEBUG
#if DEBUG
  …
#endif
```

> **Note.** `;` starts a comment in rat, so C-style `for (…; …; …)` loops cannot be written. Use `.for i in a..b`, or `.while` with `.eval`.

10.6 CPU, platform and target directives

Directive	Meaning
`.cpu name`	change the CPU: `6502`, `6502ni`, `6510`, `2a03`, `6507`, `65c02`, `45gs02`, `65816`, `spc700`, `z80`, `8080`, `6809`, `68000`, `68020`, `tms9900`, `arm2`
`.platform name [{ … }]`	change the platform; also selects its CPU
`.a8 .a16 .i8 .i16`	65816 register widths
`.setdp page`	6809 direct page; 65816 assumed D register
`.endian le|be`	byte order of data
`.include "file"`, `.once`, `.eof`	source files
`.print`, `.warn`, `.error`, `.errorif c, msg`, `.assert cond[, msg]`, `.asserterror`	messages and checks
`.break`, `.watch`	breakpoints for the VICE label file
`.syntax rat`	dialect switch (first line)

11 Operand syntax per CPU

Operands always use the CPU's native syntax, in every dialect.

CPU	Examples
65xx	`lda #<x`, `lda (zp),y`, `sta $1234,x`. Force a size with `lda.z` / `lda.a` (ACME: `lda+1` / `lda+2`)
45GS02	`lda ($10),z`, `sta [$10],z` (flat), `ldq $1234`, `adcq ($10)`, `lda ($10,sp),y`, `bbr3 $10,label`, `lbne label` (16-bit; `bne` grows by itself), `phw #$1234`, `neg a`, `map`, `eom`
65816	`lda [dp],y`, `lda $12,s`, `lda ($12,s),y`, `jml $123456`, `mvn $7e,$00`, `lda.l`, `pea #x`
SPC700	`mov a,!abs+x`, `mov a,[dp+x]`, `mov a,[dp]+y`, `mov (x)+,a`, `set1 dp.3`, `mov1 c,abs.5`, `bbs dp.2,label`, `mov dst,src`
Z80	`ld a,(ix+5)`, `ld (iy-2),$12`, `res 3,(ix+1),b`, `out (c),0`, `ex af,af'`, `ixh`/`ixl`, `sll`
8080	`mvi a,$2a`, `lxi h,table`, `mov m,a`, `push psw`, `jnz loop`, `rst 7`, `in $ff`
TMS9900	`li r1,$1234`, `mov *r1+,@$8c00`, `movb @table(r2),r3`, `ai r1,2`, `sla r3,4`, `jne loop`, `blwp @vector`, `rt` (numbers in rat syntax: `$8c00`, not `>8C00`)
ARM2	`addeqs r0,r1,r2,lsl #2`, `ldrb r0,[r1],#1`, `ldr r0,[r1,-r2]!`, `stmfd r13!,{r0-r3,r14}`, `ldmfd sp!,{pc}^`, `teqp pc,#0`, `bl label`, `swi $11`, `adr r0,label`, `ldr r0,label`; an immediate that does not encode is tried as MVN/BIC/SUB/CMN
6809	`lda <$20` (direct), `lda >$1234` (extended), `ldd ,x++`, `lda [,y]`, `leax 10,x`, `ldy label,pcr`, `pshs a,b,x`
68000/68020	`move.l #1,d0`, `lea 4(a0,d1.w*2),a1`, `(d16,pc)`, `movem.l d0-d7/a0-a6,-(sp)`, `bra.s`, `$dff180.l`, `([bd,a0],d1.w*4,od)`, `bfextu d0{4:8},d1`

12 CPUs

The ids are used with `--cpu`, `.cpu` (KickAssembler, rat) and `!cpu` (ACME). Aliases are accepted in any spelling of case, with or without `-` and a leading `_`. `rat cpus` prints this list.

Id	Byte order	Address bits	CPU	Aliases
`6502`	little	16	MOS 6502 family with undocumented opcodes (6502, 8502, 7501/8501, SALLY)	`nmos6502`, `6502x`, `8502`, `7501`, `8501`, `sally`
`6502ni`	little	16	MOS 6502, documented opcodes only	`6502noillegals`
`6510`	little	16	MOS 6510 (C64) with undocumented opcodes	`nmos6510`
`2a03`	little	16	Ricoh 2A03/2A07 (NES), no decimal mode	`2a07`, `nes`
`6507`	little	13	MOS 6507 (Atari 2600), 13-bit address bus	–
`65c02`	little	16	CMOS 65C02 (Apple IIe enhanced), no Rockwell bit instructions	`65sc02`, `w65c02`, `g65sc02`, `65sc12`
`45gs02`	little	28	MEGA65 45GS02 (CSG 4510 / 65CE02 with Q instructions and flat pointers)	`4510`, `65ce02`, `4502`, `m65`
`65816`	little	24	WDC 65C816 (SNES 5A22); register widths with `.a8`/`.a16`/`.i8`/`.i16`	`65c816`, `w65c816`, `5a22`
`spc700`	little	16	Sony SPC700 (SNES audio), Sony syntax, `!` forces absolute	`spc`
`z80`	little	16	Zilog Z80 with undocumented instructions (IXH/IXL, SLL, DD CB copies)	`z80undoc`
`8080`	little	16	Intel 8080 (IMSAI 8080), Intel mnemonics	`i8080`, `8080a`
`6809`	big	16	Motorola 6809 / 68A09 (Vectrex); `setdp` for the direct page	`68a09`, `mc6809`
`68000`	big	24	Motorola 68000 (Amiga 500/1000/2000, Macintosh)	`68k`, `mc68000`
`68020`	big	24	Motorola 68020/68EC020 (Amiga 1200)	`68ec020`, `mc68020`
`tms9900`	big	16	Texas Instruments TMS9900 (TI-99/4A)	`9900`, `tms9900a`
`arm2`	little	26	ARM2 (Acorn Archimedes), Acorn mnemonics	`arm`, `arm26`, `arm3`

13 Platforms

Ids and aliases are used with `-p`, `.platform` and `!platform`. The first format is the default; `bin`, `hex` and `srec` are allowed everywhere. `rat platforms` prints this list with the directives.

Id	Aliases	Machine	CPU	Origin	Formats	Encoding
`generic`	`none`, `raw`	no platform	6502	–	bin	ascii
`c64`	`commodore64`, `c-64`	Commodore 64	6510	$0801	prg, d64, crt	petscii_mixed
`c128`	`commodore128`	Commodore 128 (8502 side)	6502	$1C01	prg, d64	petscii_mixed
`vic20`	`vic-20`, `vc20`	Commodore VIC-20 (unexpanded)	6502	$1001	prg, d64	petscii_mixed
`pet`	`cbm`, `pet2001`	Commodore PET	6502	$0401	prg, d64	petscii_mixed
`plus4`	`c16`, `ted`, `264`, `c116`	Commodore 16 / Plus/4	6502	$1001	prg, d64	petscii_mixed
`mega65`	`m65`, `c65`	MEGA65	45gs02	$2001	prg, d64	petscii_mixed
`atari8`	`atari800`, `atarixl`, `atarixe`, `a8`, `atari`	Atari 400/800/XL/XE	6502	$2000	xex	ascii
`atari2600`	`vcs`, `a2600`, `2600`	Atari 2600 (VCS)	6507	$F000	a26	ascii
`nes`	`famicom`	Nintendo Entertainment System	2a03	$8000	nes	ascii
`snes`	`superfamicom`, `sfc`	Super Nintendo	65816	$008000	sfc	ascii
`cpc`	`amstrad`, `cpc464`, `cpc664`, `cpc6128`	Amstrad CPC	z80	$4000	amsdos	ascii
`zx81`	`ts1000`	Sinclair ZX81	z80	$4009	p	zx81
`spectrum`	`zxspectrum`, `zx`, `spectrum48`, `spectrum128`, `zx128`, `plus2`, `+2`, `speccy`	Sinclair ZX Spectrum 48K/128K/+2	z80	$8000	tap, sna	ascii
`coleco`	`colecovision`	ColecoVision	z80	$8000	bin	ascii
`msx`	`msx1`	MSX (MSX1)	z80	$4000	bin, cas	ascii
`msx2`	`msx-2`, `v9938`	MSX2	z80	$4000	bin, cas	ascii
`trs80`	`trs-80`, `trs80m1`, `trs80m3`, `model1`, `model3`	Tandy TRS-80 Model I / III	z80	$5200	cas, cmd	ascii
`imsai`	`imsai8080`, `s100`	IMSAI 8080	8080	$0100	bin	ascii
`bbc`	`bbcmicro`, `bbcb`, `bbcbplus`, `beeb`	Acorn BBC Micro Model B / B+	6502	$1900	ssd	ascii
`bbcmaster`	`master`, `master128`	Acorn BBC Master 128 (65SC12)	65c02	$1900	ssd	ascii
`archimedes`	`arc`, `a305`, `a310`, `a440`, `a3000`, `riscos`	Acorn Archimedes A305/A310/A440/A3000	arm2	$8000	bin	ascii
`apple2`	`appleii`, `apple`, `apple2plus`, `iiplus`, `a2`	Apple II / II+ / IIe	6502	$2000	dsk	ascii
`apple2e`	`apple2e_enhanced`, `iie`, `apple2c`, `iic`	Apple IIe enhanced (65C02)	65c02	$2000	dsk	ascii
`mac`	`macintosh`, `mac128`, `mac512`, `macplus`, `macse`, `mac128k`, `mac512k`	Apple Macintosh 128K/512K/Plus/SE	68000	0	bin	ascii

Id	Aliases	Machine	CPU	Origin	Formats	Encoding
`amiga`	`a500`, `a1000`, `a2000`, `a500plus`, `ocs`, `ecs`	Amiga 500/1000/2000	68000	0	hunk	ascii
`amiga1200`	`a1200`, `aga`	Amiga 1200	68020	0	hunk	ascii
`vectrex`	`gce`	Vectrex	6809	$0000	bin	ascii
`ti99`	`ti994a`, `ti-99`, `ti-99/4a`, `ti99/4a`	Texas Instruments TI-99/4A	tms9900	$6000	bin	ascii
`sim6502`	`sim`	mimic's sim6502 test machine	6502	$0400	bin	ascii

13.1 Platform directives and functions

Platforms	Directive	Meaning
`c64`	`.basicupstart address`	BASIC line `10 SYS address` at $0801 (KickAssembler `BasicUpstart2`)
`c128`, `vic20`, `pet`, `plus4`	`.basicupstart address`	BASIC line `10 SYS address` at the start of BASIC; the code continues after it
`mega65`	`.basicupstart address`	BASIC 65 line `10 BANK 0:SYS address` at $2001
`c64`	`.crt type=, name=, exrom=, game=`	CRT cartridge header for the `crt` format
all Commodore, `mega65`	`.disk [filename=, name=, id=] { [name=, type=, segments=] ... }`	build a D64 disk image (KickAssembler `.disk`)
all Commodore, `mega65`	`LoadSid()`, `LoadPicture()`, `BasicUpstart`, `BasicUpstart2`	functions and macros of the Commodore prelude
`atari8`	`.run address`	RUNAD segment: start address after loading
`atari8`	`.init address`	INITAD segment after the current segment
`atari2600`	`.bank n [, address]`	start 4K bank n (F8/F6/F4 bank switching), default address $F000
`nes`	`.ines mapper=, prg=, chr=, mirroring=h|v, battery=`	iNES header; prg in 16K units, chr in 8K units
`nes`	`.chr { ... }`	assemble the block into CHR ROM (address 0 and up)
`nes`	`.bank n [, address]`	start 16K PRG bank n
`snes`	`.header title=, map=lorom|hirom|exhirom, fast=, type=, sram=, region=, version=`	internal ROM header; the checksum is computed
`snes`	`.spc [address] { ... }`	SPC700 code assembled for `address` (default $0200), stored at the current ROM position
`cpc`	`.amsdos name=, exec=`	AMSDOS header: file name and entry point
`spectrum`	`.tap name=, exec=, loader=`	TAP output: name of the CODE block; with `exec=` (or `loader=true`) a BASIC loader CLEAR / LOAD "" CODE / RANDOMIZE USR goes first
`spectrum`	`.sna exec=, sp=, border=, im=, ei=`	SNA output: entry point (default: lowest address), stack, border colour, interrupt mode
`coleco`	`.header start [, title]`	cartridge header at $8000 ($AA55, pointers, start address)
`msx`, `msx2`	`.header init`	cartridge header (`AB`, INIT address, 12 zero bytes)
`msx`, `msx2`	`.cas name=, exec=`	CAS output: name and start address of the BLOAD file
`trs80`	`.cas name=, exec=`	CAS output: name and entry point of the SYSTEM tape

Platforms	Directive	Meaning
`trs80`	`.cmd exec`	CMD output: entry point of the load module
`bbc`, `bbcmaster`	`.ssd name=, exec=, title=, boot=`	SSD output: DFS file name, execution address, disc title; `boot=true` names the file `!BOOT` and sets `*OPT 4,2` (Shift+BREAK runs it)
`apple2`, `apple2e`	`.dsk name=, volume=, type=, locked=`	DSK output: catalog name, volume number (254), DOS file type byte ($04 binary)
`amiga`, `amiga1200`	`.section name, code|data|bss`	start a hunk; absolute longwords are relocated (RELOC32)
`vectrex`	`.header title [, music]`	cartridge header (`g GCE` copyright, music, title)
`ti99`	`.header title, start`	cartridge header at $6000 with one program entry (name in the selection list, start address)

14 Output formats

Select a format with `-f`, with `.output "file", format` or by the platform's default. `rat formats` prints this list.

Format	Extension	Contents	Platforms
`bin`	`.bin`	raw binary, lowest to highest written address	all
`hex`	`.hex`	Intel HEX	all
`srec`	`.s19`	Motorola S-records	all
`prg`	`.prg`	Commodore PRG (load address + data)	c64, c128, vic20, pet, plus4, mega65
`d64`	`.d64`	1541 disk image	c64, c128, vic20, pet, plus4, mega65
`crt`	`.crt`	C64 CRT cartridge	c64
`xex`	`.xex`	Atari 8-bit XEX (segments, RUNAD/INITAD)	atari8
`a26`	`.a26`	Atari 2600 cartridge (2K/4K, F8/F6/F4 banks)	atari2600
`nes`	`.nes`	iNES ROM (header + PRG + CHR)	nes
`sfc`	`.sfc`	SNES ROM (LoROM/HiROM/ExHiROM, checksum fixed)	snes
`amsdos`	`.bin`	Amstrad CPC binary with AMSDOS header	cpc
`p`	`.p`	ZX81 .P file (memory from $4009)	zx81
`tap`	`.tap`	ZX Spectrum tape (CODE block, optional BASIC loader)	spectrum
`sna`	`.sna`	ZX Spectrum 48K snapshot (starts at the entry point)	spectrum
`cas`	`.cas`	cassette image: MSX BLOAD file, TRS-80 SYSTEM tape	msx, msx2, trs80
`cmd`	`.cmd`	TRS-80 /CMD load module	trs80
`dsk`	`.dsk`	Apple II DOS 3.3 disk image with the program as a binary file	apple2, apple2e
`ssd`	`.ssd`	BBC Micro DFS disk image (40 tracks) with the program as a file	bbc, bbcmaster
`hunk`	`.exe`	Amiga hunk executable (with RELOC32)	amiga, amiga1200

15 Command line reference

15.1 rat build SOURCE

Option	Meaning
`-o, --output FILE`	Output file (default: source name with the format's extension).
`--cpu CPU`	CPU; overrides the platform default.
`-p, --platform PLATFORM`	Target platform.
`-s, --syntax DIALECT`	Dialect of the main file: `acme`, `kick`, `rat`.
`-f, --format FORMAT`	Output format.
`-D, --define NAME[=VALUE]`	Define a symbol (also in `cmdLineVars`); repeatable.
`-I, --include DIR`	Include directory; repeatable.
`--listing FILE`	Write a listing.
`--vice-labels FILE`	Write VICE labels (readable by mimic's debugger).
`--sym FILE`	Write a KickAssembler `.sym` file.
`--labels FILE`	Write an ACME label list.
`--passes N`	Maximum number of passes (default 20).
`--json`	Print a summary of outputs and symbols as JSON.

15.2 rat disasm FILE

Option	Meaning
`--cpu CPU`	CPU (required).
`--org ADDR`	Load address (default: from a PRG header, else 0).
`--skip N`	Skip bytes at the start (default 0).
`--prg`	Treat the file as a PRG (2-byte load address).
`--a16, --i16`	65816: 16-bit accumulator, 16-bit index registers.

15.3 rat convert FILE

Option	Meaning
`--to DIALECT`	Target dialect: `acme`, `kick`, `rat` (required).
`--from DIALECT`	Source dialect (default: detected).
`-o, --output FILE`	Output file (default: standard output).

15.4 rat serve

Option	Meaning
`--bind ADDR`	Address and port (default `127.0.0.1:8080`).

16 REST API reference

16.1 POST /assemble

JSON body, or multipart/form-data with file parts and text fields.

Field	Meaning
`files`	Files by path: plain text, or `{"base64": "..."}` for binaries. Example: `{"main.asm": "...", "gfx/logo.bin": {"base64": "..."}}`.
`main`	Path of the main source in `files`.
`cpu`, `platform`, `syntax`, `format`	Optional; as on the command line.
`defines`	Optional object of `-D` defines, name to value.
`listing`	Optional; `true` adds the listing and the source line of every address.

In a multipart form the file name of a part is its path; the text fields are `main`, `cpu`, `platform`, `syntax`, `format`, `listing` and `define` (`NAME=VALUE`, repeatable).

Response field	Meaning
`ok`, `passes`, `cpu`, `platform`	Result, number of passes, the CPU and platform used.
`outputs`	List of `{name, format, size, base64}`.
`diagnostics`	List of `{severity, message, file, line, column, rendered}`; severity is error, warning or info, `rendered` the text with the source line.
`symbols`	List of `{name, value, label}`.
`prints`	The `.print` output.
`listing`, `lines`	With `listing`: the listing text and `{file, line, addr, size}` for every line.
`vice_labels`	VICE monitor labels.

16.2 POST /disassemble

Field	Meaning
`cpu`	CPU id.
`org`	Load address (default 0).
`base64`	The bytes.
`a16`, `i16`	65816: 16-bit accumulator, 16-bit index registers.

The response holds `lines` (`{addr, bytes, text}`) and `source`, the rat-dialect source.

16.3 POST /convert

Field	Meaning
`source`	The source text.
`name`	Optional file name; its extension helps detect the dialect.
`from`	Optional source dialect.
`to`	Target dialect.

The response holds `source` and `warnings`.

16.4 GET /cpus, /platforms, /formats, /health

`/cpus` lists id, aliases, description, byte order, address bits and the mnemonics of each CPU; `/platforms` id, aliases, name, CPU, formats, default origin, directives and functions; `/formats` name, extension, description and platforms (`*` means all). An error response is `{"error": "..."}`.

17 Known gaps

- KickAssembler: `.plugin` (Java), `.modify`/`.filemodify` and segment modifiers, `createFile`, and label arrays of loops (`loop[i].label`) are not supported. The `dtv` CPU is missing because mimic does not emulate it.
- `65c02` has no Rockwell/WDC bit instructions (`RMB`, `SMB`, `BBR`, `BBS`, `STP`, `WAI`), like mimic's 65C02 (the NCR/GTE part of the Apple IIe).
- There is no separate 4510 or 65CE02 CPU: `!cpu 65ce02`, `4510` and `4502` select the 45GS02, a superset of both.
- MSX and MSX2 share one platform definition; there are no MSX2-specific directives or a disk image writer.
- Archimedes, TI-99/4A and IMSAI output is raw code (`bin`, `hex`, `srec`): there is no RISC OS ADFS image, TI disk or cassette writer.
- Macintosh output is raw code; there is no resource fork or MFS disk writer.
- Amiga: there is no ADF or hard disk (HDF) image writer; the hunk executable is copied to a disk with other tools.
- mimic's own Logophor 16 (CPU ADA) is not a rat target; it has its assembler in mimic.
- Converting KickAssembler script code (lists, structs, functions, pseudocommands, segments) to ACME leaves `TODO(rat convert)` comments.

18 How it works

```
crates/
  rat-core      spans, diagnostics, tokens, configurable lexer, statement IR
  rat-cpu       CPU backends (encoder + disassembler per CPU)
  rat-syntax    ACME and Kick/rat parsers -> shared IR
  rat-platform  platform registry, Commodore preludes, SID/picture loading
  rat-output    prg, xex, nes, sfc, a26, p, amsdos, hunk, crt, d64, tap, sna,
                dsk, ssd, cas, cmd, hex, srec, symbol files
  rat-asm       the multi-pass engine, script values, converter
  rat-server    REST API
  rat-cli       the `rat` program
tests/          round trips against mimic, corpus with golden outputs
tools/          the builder of this guide
```

18.1 Passes

The engine runs the program once per pass. A forward reference uses the value from the previous pass. Operand sizes, such as zero page versus absolute or short versus long branches, can only grow, so the passes converge. Assembly stops when no symbol changes. If values never settle, the undefined symbols are reported.

18.2 Amiga relocation

Hunk executables are relocated by assembling again with one hunk moved: longwords that move with it become RELOC32 entries.

18.3 Testing

```
cargo test --workspace --release
```

- **Round trips against mimic.** Every 6502, 65C02, 45GS02, 65816, SPC700, 8080, Z80 and 6809 opcode, every 68000 and TMS9900 opcode word, a third of the 68020 space and every ARM2 condition, class and flag combination is disassembled with mimic's disassembler, assembled with rat, disassembled again and compared.
- **Corpus.** Sources for the platforms are assembled against golden outputs and converted to every dialect.
- **Media.** Tape and disk images are read back with mimic's media readers.
- **REST.** JSON and multipart assembly, disassembly, conversion and the OpenAPI document.

18.4 Building this guide

```
pip install reportlab
python tools/user_guide.py              # -> docs/rat-user-guide-en.pdf, -de.pdf
```